

CPSC 375 High-Performance Computing

Lecture 6

Semaphores

Linux IPC

- Pipes
- Message Queues
- Shared memory
- **Semaphores**
- Signals
- Sockets

The Critical Section Problem

- A segment of code where shared variables, files, or memory regions are accessed.
- Only one process should be in its critical section.
(*mutual exclusion*)
- If no process is in the critical section, any process requesting entry must be allowed.
- A limit on how many other processes can be in the critical section.

Race Conditions

- *Atomic*: non-interruptible or indivisible operation
 - e.g., `counter++`, is not atomic and require the CPU to load, increment, and then store the value.
 - OS may pause a process in the middle of these three steps to give CPU time to another task.
 - If Process B completes a full increment cycle while Process A is paused, Process A will overwrite B's work when it resumes, leading to a lower final count (Homework 5).
- A secondary control, like *semaphores*, is required.

Semaphores

- Data structure that is shared by several processes.
- Used to synchronize operations when multiple processes access a command, non-shareable resource.
- A type of generalized *lock*
 - Defined by Edsger Dijkstra in the last 60s
 - Main synchronization primitives used in Linux
 - A positive integer value acting as a signal for processes to proceed or wait, preventing uncoordinated access to the critical section.
 - Unlike simple user-space flags, semaphores are managed by the kernel.
 - Can manage a pool of multiple resources by allowing a set number of processes to enter a region before blocking the rest.

Semaphores

- Two operations
 - **P()**: an atomic operation that waits for semaphore to become positive, then *decrements* it by 1
 - **V()**: an atomic operation that *increments* semaphore by 1 and wakes up a waiting process at P(), if any.
- Two types
 - *Binary semaphores*: can only take values of 0 or 1
 - *Counting semaphores*: initialized to a value $N > 1$ to manage a pool of identical resources, allowing N processes to work simultaneously before blocking the $N+1$ th.

Semaphores vs. Integers

- No negative values
- Only operations are P() and V()
 - Cannot read or write semaphore values
 - Except at the initialization times
- Operations are atomic
 - Two P() calls cannot decrement the value below zero
 - A sleeping process at P() cannot miss a wakeup from V()

Binary Semaphores

- A binary semaphore is initialized to 1
- P() waits until the value is 1
 - Then set it to 0
- V() sets the value to 1
 - Wakes up a process waiting at P(), if any

Example

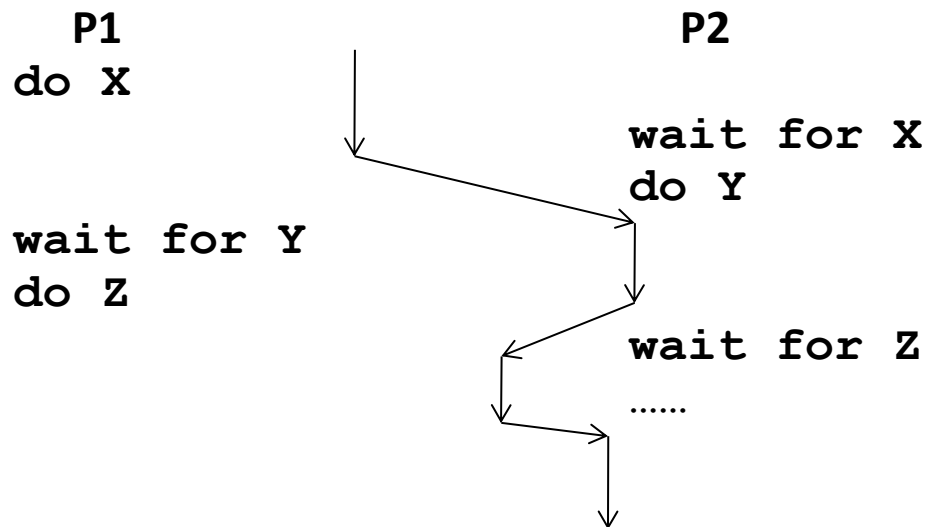
Mutual exclusion: implements a lock function

- Semaphore has an initial value of 1
- P() is called before a critical section
- V() is called after the critical section

```
semaphore s = 1;  
P(s);  
// start of critical section  
...  
// end of critical section  
V(s);
```

Example, cont'd

Scheduling: Enforcing some order between processes



Example, cont'd

Scheduling

- Semaphore usually has an initial value of 0

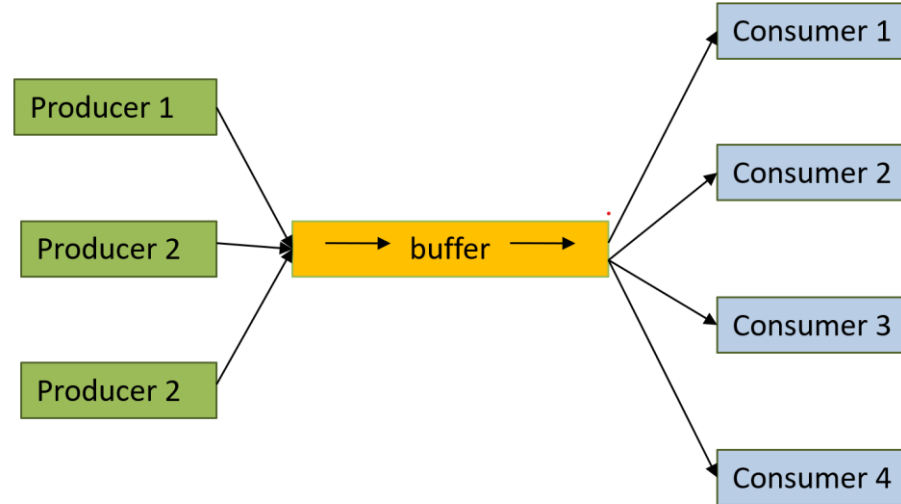
```
semaphore s1 = 0;
```

```
semaphore s2 = 0;
```

```
P1() {  
    write(x);  
    V(s1);  
    P(s2);  
    read(y);  
}
```

```
P2() {  
    P(s1);  
    read(x);  
    write(y);  
    V(s2);  
}
```

Producer Consumer Problem with Bounded Buffer



- Constraints
 - The consumer must wait if buffers are empty.
 - The producer must wait if buffers are full.
 - Only one process can manipulate the buffer at a time (mutual exclusion)

Producer Consumer Problem with Bounded Buffer

- Each constraint needs a semaphore

```
semaphore mutex = 1;  
semaphore nFreeBuffers = N;  
semaphore nLoadedBuffers = 0;
```

Producer Consumer Problem with Bounded Buffer

```
semaphore mutex = 1;  
semaphore nFreeBuffers = N;  
semaphore nLoadedBuffers = 0;
```

```
Producer() {
```

```
    P(mutex);  
    // put 1 item in the buffer  
    V(mutex);
```

```
}
```

```
Consumer() {
```

```
    P(mutex);  
    // take 1 item from the  
    // buffer  
    V(mutex);
```

```
}
```

Producer Consumer Problem with Bounded Buffer

```
semaphore mutex = 1;  
semaphore nFreeBuffers = N;  
semaphore nLoadedBuffers = 0;
```

```
Producer() {  
    P(nFreeBuffers);  
    P(mutex);  
    // put 1 item in the buffer  
    V(mutex);  
  
}
```

```
Consumer() {  
    P(nLoadedBuffers);  
    P(mutex);  
    // take 1 item from the  
    // buffer  
    V(mutex);  
  
}
```

Producer Consumer Problem with Bounded Buffer

```
semaphore mutex = 1;
semaphore nFreeBuffers = N;
semaphore nLoadedBuffers = 0;

Producer() {
    P(nFreeBuffers);
    P(mutex);
    // put 1 item in the buffer
    V(mutex);
    V(nLoadedBuffers);
}
```

```
Consumer() {
    P(nLoadedBuffers);
    P(mutex);
    // take 1 item from the
    // buffer
    V(mutex);
    V(nFreeBuffers);
}
```

POSIX Semaphores

To define a semaphore object, use

```
sem_t s;
```

Initializing a Semaphore

```
int sem_init(sem_t *sem, int pshared,  
             unsigned int value);
```

- `sem` **points to a semaphore object to initialize**
- `pshared` is a flag indicating whether or not the semaphore should be shared with `fork()`ed processes.
- `value` is an initial value to set the semaphore
- **Example:**

```
sem_init(&sem_name, 0, 10);
```

Waiting on a Semaphore

```
int sem_wait(sem_t *sem);
```

- **Example:**

```
sem_wait(&sem_name);
```

- If the value of the semaphore is negative, the calling process blocks.
- One of the blocked processes wakes up when another process calls `sem_post`.

Incrementing a Semaphore

```
int sem_post(sem_t *sem) ;
```

- **Example:**

```
sem_post(&sem_name) ;
```

- It increments the value of the semaphore and
- Wakes up (signals) a blocked process waiting on the semaphore, if any.

Accessing a Semaphore

```
int sem_getvalue(sem_t *sem, int *valp);
```

- Gets the current value of `sem` and places it in the location pointed to by `valp`.

Destroying a Semaphore

```
int sem_destroy(sem_t *sem);
```

- Destroys the semaphore
- No processes should be waiting on the semaphore.

An Example

proc1

```
a = data;  
a++;  
data = a;
```

proc2

```
b = data;  
b--;  
data = b;
```

The Example with a Binary Semaphore

```
sem_t mutex;  
sem_init(&mutex, 0, 1);
```

proc1

```
sem_wait(&mutex);  
a = data;  
a++;  
data = a;  
sem_post(&mutex);
```

proc2

```
sem_wait(&mutex);  
b = data;  
b--;  
data = b;  
sem_post(&mutex);
```

The Readers/Writers Problem

- Models access to a database, such as an airline reservation system
 - Multiple readers (customers) want to read the database — access schedules, flight availability, etc.
 - Multiple writers (the airline) want to write the database — update the schedules
- It is acceptable to have multiple readers at the same time
 - But while a writer is writing to (updating) the database, no one else can access the database — either to read or to write
- Two versions:
 - Readers have priority over writers
 - Writers have priority over readers
 - In most solutions, the non-priority group can *starve*

Notes on Readers

- Needs mutually exclusive access while manipulating “readers” variable.
- Does not need mutually exclusive access while reading database.
- If this reader is the first reader, it has to wait if there is an active writer.
- If other readers come along while the first one is waiting, they also wait.
- If other readers come along while the first one is actively reading the database, they can also get in to read the database.
- When the last reader finishes, if there are waiting writers, it must wake one up.

Notes on Writers

- If there is an active writer, this writer has to wait.
- If there are active readers, this writer has to wait.
- The writer only gets in to write to the database when there are no other active readers or writers
- When the writer finishes, it wakes up someone (either a reader or a writer — it's up to the CPU scheduler)
- If a reader gets to go next, then once it goes through the “V(mutex)” and starts reading the database.

RWP with Readers Priority

Initialization:

```
semaphore mutex = 1; /* for mutual exclusion */  
semaphore write = 1; /* for mutual exclusion & synchronization */  
int readers = 0; /* number of readers */
```

Reader:

```
P(mutex);  
readers++;  
if (readers == 1)  
    P(write);  
V(mutex);  
read_database();  
P(mutex);  
readers--;  
if (readers == 0)  
    V(write);  
V(mutex);
```

Writer:

```
P(write);  
write_database();  
V(write);
```

